

Hardware Profiling Manual

Tables 4.3, 4.4 and 4.5

Reduced Precision Hardware Acceleration of Nested Multi-Level Monte Carlo using CPUs

Mohammad Saifullah

September 2026

This note records how the two kernel-performance tables were produced: which source was compiled, how it was run, and which counter event each printed number came from. It is a record of method rather than of results, written so that any row can be traced to a line of raw output or reproduced from scratch.

All source and all raw output referred to below are in the repository <https://github.com/Saif2024C/mlmc-reduced-precision>. Paths are given relative to its root.

1 Platform

Host	<code>mimic.maths.ox.ac.uk</code>
CPU	Intel Xeon Gold 6538Y+ (Emerald Rapids), AVX-512 with the FP16 extension
Compiler	<code>icpx</code> (Intel oneAPI)
Measurement	Linux <code>perf stat</code> , whole-process counters
Threads	1 (serial build; no <code>-qopenmp</code>)

Both tables describe the same object: the level-0 kernel of the basket European estimator, which draws five correlated Brownian increments, advances each asset by one Milstein step, forms the weighted basket, and evaluates $e^{-rT} \max(B_T - K, 0)$. Level 0 was chosen because it is a single timestep, so the measurement is of the arithmetic itself and not of a loop over levels.

2 Source and build

One source file provides both precisions, selected at run time by `--kernel`:

```
src/AVX-512/performance_analysis/kernel_bench_basket_l0.cpp
```

The FP32 path holds the asset state in `__m512` (16 lanes); the FP16 path holds it in `__m256h` (16 lanes of half) with Kahan compensation on the running state. Both consume the same Philox stream, so the two kernels perform the same work on the same numbers and differ only in the precision of the path arithmetic.

Build (from `src/AVX-512/`):

```
source /opt/intel/oneapi/setvars.sh
icpx -O3 -march=native -std=c++17 \
  --gcc-install-dir=/usr/lib/gcc/x86_64-linux-gnu/13 \
  performance_analysis/kernel_bench_basket_l0.cpp -o kb_basket_l0
```

equivalently make `kb_basket_l0`, which issues the same line; see `src/AVX-512/Makefile`. The `--gcc-install-dir` flag pins `icpx` to the gcc-13 toolchain. Without it `icpx` selects gcc-14, whose directory exists on these nodes but whose `libstdc++` headers do not, and the compilation fails with `C++ header location not resolved`.

3 Table 4.3: timings across the driver’s sample counts

What was run

```
make kbench REPS=9
# expands to: ./kb_basket_l0 --sweep --reps 9
```

Raw output: `outputs/avx512/kernel/basket/basket_l0_eps_sweep.txt`.

Protocol

The sample counts N_0 are not chosen for the benchmark. They are the level-0 entries of the N_l column of the MLMC complexity table for the basket European payoff, so each row is a size the estimator actually requests at that accuracy. They are compiled into the benchmark as a table and must be re-derived if the driver’s `Eps[]` array or the model constants change.

Each timing is the *fastest* of nine repetitions, after one warm-up repetition. The fastest is taken rather than the mean because `mimic` is shared: competing work on the machine can only make a repetition slower, never faster, so all nine overstate the true cost by varying amounts and the fastest is the one that was interfered with least. Averaging would build that interference into the result instead of discarding it. Both precisions are timed within a single run of the benchmark, so the two are measured under the same machine conditions.

The kernel also returns `chk`, the Monte Carlo mean $N^{-1} \sum_p P^{(p)}$, which is printed alongside each timing as a correctness check that the two precisions price the same option.

Reading the table

Columns `ms` are wall clock; `ns/path` is that divided by N_0 ; the ratio column is $T_{\text{FP32}}/T_{\text{FP16}}$. No counters are involved in this table.

4 Table 4.4: hardware counters

What was run

```
make kbench_allcounters BENCH=kb_basket_l0
```

Raw output: `outputs/avx512/kernel/basket_l0_allcounters.txt`.

Why four passes rather than one

The processor has only a handful of hardware registers for counting events, fewer than the fourteen needed here (Table 1). Asked for all of them at once, `perf` runs each event for a fraction of the execution and scales the result up to a full-run estimate; it reports the fraction as a percentage beside each line. Every figure would then be an extrapolation. Splitting the events into four groups, each small enough to fit in the registers available, means every number is counted over the whole run instead.

The cost of splitting is that the four groups require four separate executions, so dividing a quantity from one by a quantity from another is only valid if the two ran under comparable conditions. All eight executions (four passes $\times$ two precisions) are therefore issued by a single `make` target in one session. The clock across the eight is 2.648–2.655 GHz, a spread of 0.3%, which is the evidence that they did.

Event	Used for	Pass
<code>task-clock</code>	clock frequency	all
<code>cycles</code>	cycles row; IPC denominator	all
<code>instructions</code>	IPC numerator	all
<code>fp_arith_inst_retired.512b_packed_single</code>	FP32 arithmetic	1
<code>fp_arith_inst_retired.256b_packed_single</code>	FP32 arithmetic	1
<code>fp_arith_inst_retired2.512b_packed_half</code>	FP16 arithmetic	1
<code>fp_arith_inst_retired2.256b_packed_half</code>	FP16 arithmetic	1
<code>L1-dcache-loads</code>	loads row; load-port pressure	2, 3, 4
<code>L1-dcache-load-misses</code>	L1 miss rate	2
<code>l2_rqsts.miss</code>	L2 misses	2
<code>L1-dcache-stores</code>	stores row; store-port pressure	3, 4
<code>mem_inst_retired.all_loads</code>	non-memory IPC	4
<code>mem_inst_retired.all_stores</code>	non-memory IPC	3, 4
<code>resource_stalls.sb</code>	collected, not reported	3

Table 1: The fourteen events, what each contributes, and which pass collects it. The first three are repeated in every pass so the passes can be checked against one another.

The four passes

Each pass runs `./kb_basket_10 --kernel {fp32,fp16}` under `perf stat` with the events below. `task-clock`, `cycles` and `instructions` are repeated in every pass, which is what makes the consistency check above possible.

Pass 1, instruction mix.

```
perf stat -e task-clock,cycles,instructions,\
fp_arith_inst_retired.512b_packed_single,\
fp_arith_inst_retired.256b_packed_single,\
fp_arith_inst_retired2.512b_packed_half,\
fp_arith_inst_retired2.256b_packed_half \
./kb_basket_10 --kernel fp32
```

FP16 arithmetic is counted by `fp_arith_inst_retired2.*`, a separate event group from the FP32 `fp_arith_inst_retired.*` rather than a variant of it. Note the trailing 2. Without it the FP16 kernel registers as having almost no vector activity, since its work is invisible to the `_single` counters. Both groups are collected for both kernels so the comparison is like for like, and each kernel duly reads zero on the group belonging to the other.

Pass 2, cache behaviour.

```
perf stat -e task-clock,cycles,instructions,\
L1-dcache-loads,L1-dcache-load-misses,l2_rqsts.miss \
./kb_basket_10 --kernel fp32
```

Pass 3, store side.

```
perf stat -e task-clock,cycles,instructions,\
L1-dcache-loads,L1-dcache-stores,mem_inst_retired.all_stores,\
resource_stalls.sb \
./kb_basket_l0 --kernel fp32
```

Pass 4, memory share of the instruction stream.

```
perf stat -e task-clock,cycles,instructions,\
mem_inst_retired.all_loads,mem_inst_retired.all_stores,\
L1-dcache-loads,L1-dcache-stores \
./kb_basket_l0 --kernel fp32
```

This pass exists to separate memory instructions from the rest of the stream, for the non-memory IPC of Table 4.5.

5 Where each row of Table 4.4 comes from

All figures are at $N = 4,194,304$ paths, single threaded.

Row	Source	Derivation
Time per path	benchmark, not perf	best of 5 repetitions, $\div N$
Cycles	pass 4, cycles	as reported
FP-arithmetic	pass 1	512b_packed + 256b_packed
IPC	pass 4	instructions $\div$ cycles
Clock frequency	pass 4	cycles $\div$ task-clock
L1 loads	pass 2, L1-dcache-loads	as reported
L1 stores	pass 3, L1-dcache-stores	as reported
Load-port pressure	pass 2	loads $\div$ 2
Store-port pressure	pass 3	stores $\div$ 1
L1 miss rate	pass 2	load-misses $\div$ loads
L2 misses	pass 2, l2_rqsts.miss	as reported

Worked example: the FP-arithmetic row

From pass 1 of `basket_l0.allcounters.txt`:

Event	FP32	FP16
fp_arith_inst_retired.512b_packed.single	115,703,808	0
fp_arith_inst_retired.256b_packed.single	144,629,760	0
fp_arith_inst_retired2.512b_packed.half	0	32,870,400
fp_arith_inst_retired2.256b_packed.half	0	101,240,832
Total	260,333,568	134,111,232

giving the 260.3 M and 134.1 M of Table 4.4 and a ratio of $1.94\times$. The zeros confirm that each kernel is executing in the precision intended: neither issues a single instruction of the other's type.

6 Table 4.5: non-memory IPC

No separate run is needed. Every figure comes from pass 4, which was collected for this purpose: it is the only pass carrying loads, stores, instructions and cycles together, so all

four are counted in one execution and the derived rows involve no cross-pass division.

Writing I , L , S , C for the retired instruction, load, store and cycle counts, the derived rows are

$$I_{\text{non-mem}} = I - L - S, \quad \text{memory share} = \frac{L + S}{I}, \quad \text{IPC}_{\text{non-mem}} = \frac{I - L - S}{C}.$$

The four counters read directly from pass 4 of `basket_l0.allcounters.txt`:

Counter	FP32	FP16
instructions	2,192,924,984	896,196,278
L1-dcache-loads	658,039,820	126,439,500
L1-dcache-stores	231,149,811	25,790,918
cycles	1,446,102,331	700,835,160

giving $I_{\text{non-mem}} = 1303.7\text{ M}$ against 744.0 M , memory shares of 40.5% and 17.0%, and the two IPC figures 0.90 and 1.06 against 1.52 and 1.28 on the whole stream.

7 Reproducing the tables

```
cd src/AVX-512
source /opt/intel/oneapi/setvars.sh
make kb_basket_l0
make kbench REPS=9 # Table 4.3
make kbench_allcounters BENCH=kb_basket_l0 # Tables 4.4 and 4.5
```

The second target is what issues the four `perf stat` commands listed above. It runs each of them twice, once with `--kernel fp32` and once with `--kernel fp16`, then the timing block, giving the nine executions in one session. It is not a single combined invocation, so no event is multiplexed. Output is written under `outputs/avx512/kernel/`. The same procedure applied to the scalar Asian kernel produces the appendix tables; substitute `kb_scalar_asian_l0` for `kb_basket_l0` throughout, with raw output in `outputs/avx512/kernel/kb_scalar_asian_l0.allcounters.txt`.

8 Limitations

1. The counters are whole-process, so they include the RNG, process start-up and the accumulation loop, not the Milstein arithmetic in isolation. Both precisions carry the same surrounding work, so the comparison between them is unaffected, but the absolute counts are larger than the arithmetic alone would give.
2. Time per path comes from the benchmark's own timer, not from `perf`. The `cycles` row and the time row are therefore independent measurements of the same quantity; they agree to within 0.5%: the cycle counts fall by $2.06\times$ and the wall clock by $2.07\times$.
3. `mimic` is a shared machine. The load average at the time of the run is recorded in the header of the raw output. Taking the fastest repetition mitigates this for Table 4.3; the figures in Table 4.4 are counts rather than timings and are insensitive to it.